

Metodi matematici per il Machine Learning

Leonardo Ganzaroli

Indice

1	Capitolo I	3
1.1	Prodotti e spazi	3
1.1.1	Matrice $\times$ Vettore	3
1.1.2	Matrice $\times$ Matrice	4
1.1.3	Sottospazi	5
1.2	Eliminazione e $A = LU$	6
1.3	Ortogonalità	8
1.4	Autovalori e autovettori	9
1.4.1	Teoria	9
1.4.2	Calcolo e diagonalizzazione	10
1.5	Matrici Simmetriche Definite Positive	12
1.6	Norme	13
1.7	SVD	15
1.8	Analisi dei componenti principali	17
2	Capitolo II	18
2.1	Minimi quadrati	18
3	Capitolo III	19
3.1	$\Delta A \rightarrow \Delta A^{-1}$	19
3.2	Splitting $\ell_1 + \ell_2$	20
4	Capitolo VI	21
4.1	Problemi di minimo e convessità	21
4.2	Moltiplicatori di Lagrange	22
4.3	Discesa del gradiente	23
5	Capitolo 2	24
6	Regressione	25
7	Deep Learning	26
7.1	Reti neurali	26
7.2	CNN	29
7.2.1	Tensori e operazioni	29

7.2.2	Funzionamento generale	31
7.2.3	Casi d'uso e metriche	32
7.3	Problemi comuni	34
7.4	Altri argomenti	35

1 Capitolo I¹

1.1 Prodotti e spazi

1.1.1 Matrice $\times$ Vettore

La moltiplicazione di una matrice per un vettore si può fare con 2 procedimenti:

$$\begin{aligned} Ax = \begin{bmatrix} 2 & 3 \\ 2 & 4 \\ 3 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= \begin{bmatrix} 2x_1 + 3x_2 \\ 2x_1 + 4x_2 \\ 3x_1 + 7x_2 \end{bmatrix} && (\text{Prodotto scalare}) \\ &= x_1 \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix} + x_2 \begin{bmatrix} 3 \\ 4 \\ 7 \end{bmatrix} && (\text{Combinazione lineare delle colonne}) \end{aligned}$$

Definizione Lo spazio delle colonne di una matrice è l'insieme di tutte le combinazioni lineari delle sue colonne:

$$C(A) = \text{span}\{a_1, \dots, a_n\}$$

Quindi $b \in C(A) \iff Ax = b$ ha almeno una soluzione.

Definizione Un insieme di colonne è detto linearmente indipendente se l'unica combinazione che produce il vettore nullo è quella in cui tutti coefficienti sono pari a 0.

Definizione Le colonne linearmente indipendenti di una matrice costituiscono la base dello spazio delle colonne.

Definizione Il rango di una matrice è il numero di colonne linearmente indipendenti.

Definizione Creando la matrice C con le colonne indipendenti di una matrice A posso costruire una seconda matrice tale che $CR = A$:

$$A = \begin{bmatrix} 1 & 3 & 8 \\ 1 & 2 & 6 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 1 & 2 \\ 0 & 1 \end{bmatrix}^C \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \end{bmatrix}^R$$

Le righe di R rappresentano una base per lo spazio delle righe di A .

Teorema 1 *Il numero di colonne indipendenti è uguale a quello di righe indipendenti.*

¹I capitoli in numeri romani fanno riferimento a “Linear Algebra and Learning from Data” di G. Strang.

1.1.2 Matrice $\times$ Matrice

La situazione è analoga per il prodotto tra 2 matrici:

$$\begin{aligned} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \quad (\text{Prodotto riga-colonna}) \\ &= \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \end{bmatrix} + \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} \begin{bmatrix} b_{21} & b_{22} \end{bmatrix} \quad (\text{Somma di prodotti colonna-riga}) \end{aligned}$$

Nel secondo caso un singolo elemento della somma è uv^T che risulta essere una matrice ($m \times p$). Dato che le colonne di questa sono multipli di u e le righe a loro volta sono multipli di v^T , si ha $\text{Rank}(uv^T) = 1$ (se $u \neq 0$ e $v \neq 0$).

Prendendo CR dall'esempio precedente:

$$\begin{bmatrix} 1 & 3 \\ 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 3 & 6 \\ 0 & 2 & 4 \\ 0 & 1 & 2 \end{bmatrix}$$

Definizione Una matrice è simmetrica (S) se: $S^T = S$.

Definizione Una matrice è ortogonale (Q) se: $Q^T = Q^{-1}$.

Alcune fattorizzazioni importanti:

- $A = LU$
- $A = QR$
- $A = X\Lambda X^{-1}$
- $S = Q\Lambda Q^T$
- $A = U\Sigma V^T$

1.1.3 Sottospazi

Definizione Un sottospazio vettoriale di uno spazio vettoriale è un sottoinsieme di questo che è a sua volta uno spazio vettoriale, cioè è chiuso rispetto alla somma di vettori e al prodotto per uno scalare. Uno spazio è sempre sottospazio di sé stesso.

Oltre a quello delle righe e quello delle colonne una matrice $A_{m \times n}$ definisce altri 2 sottospazi:

1. **Null**

$$N(A) = \{x \in \mathbb{R}^n : Ax = 0\}$$

2. **Null** **sx**

$$N(A^T) = \{y \in \mathbb{R}^m : A^T y = 0\}$$

Teorema 2

Se ho $A \in \mathbb{R}^{m \times n}$ di rango r allora:

$$\dim(C(A)) = r, \dim(C(A^T)) = r, \dim(N(A)) = n - r, \dim(N(A^T)) = m - r$$

Per matrici compatibili valgono le seguenti proprietà:

$$\text{Rank}(AB) \leq \text{Rank}(A)$$

$$\text{Rank}(AB) \leq \text{Rank}(B)$$

$$\text{Rank}(A + B) \leq \text{Rank}(A) + \text{Rank}(B)$$

$$\text{Rank}(A^T A) = \text{Rank}(A) = \text{Rank}(A A^T)$$

$$\text{Rank}(A) = \text{Rank}(A^T)$$

1.2 Eliminazione e $A = LU$

Definizione Una matrice quadrata è triangolare superiore se tutti gli elementi sotto la diagonale principale sono 0, equivalentemente è triangolare inferiore se tutti gli elementi sopra la diagonale principale sono 0.

Definizione Una matrice di permutazione P è una matrice quadrata che in ogni riga e colonna ha un solo 1, mentre tutti gli altri elementi sono 0.

Il sistema:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

può essere riscritto come $Ax = b$, dove:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Definizione L'eliminazione di Gauss è un algoritmo che applica operazioni elementari tra righe alla matrice A trasformandola in una triangolare superiore U . In ogni passo si sceglie un elemento (pivot) che elimina gli elementi sottostanti nella sua colonna. Nel caso $Ax = b$, le stesse operazioni vengono applicate anche a b , ottenendo un sistema equivalente più semplice da risolvere.

A questo punto creando la matrice triangolare inferiore L con i coefficienti moltiplicativi usati nel procedimento si ottiene la fattorizzazione LU . Se durante l'eliminazione un pivot è nullo (o troppo piccolo), si possono scambiare 2 righe con una matrice di permutazione, in questo caso la fattorizzazione diventa $PA = LU$.

Esempio:

$$\begin{bmatrix} 2 & 1 & -1 \\ 4 & 3 & -1 \\ -2 & 2 & 3 \end{bmatrix}$$

$$\text{Primo pivot: } a_{1,1} \rightarrow m_{2,1} = \frac{4}{2} = 2, \quad m_{3,1} = \frac{-2}{2} = -1$$

$$R_2 = R_2 - 2R_1, \quad R_3 = R_3 + R_1$$

$$\begin{bmatrix} 2 & 1 & -1 \\ 0 & 1 & 1 \\ 0 & 3 & 2 \end{bmatrix}$$

Secondo pivot: $a_{2,2} \rightarrow m_{3,2} = \frac{3}{1} = 3$

$$R_3 = R_3 - 3R_2$$

$$U = \begin{bmatrix} 2 & 1 & -1 \\ 0 & 1 & 1 \\ 0 & 0 & -1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 3 & 1 \end{bmatrix}$$

Esempio con permutazione:

$$\begin{bmatrix} 0 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 2 \\ 0 & 2 & 1 \end{bmatrix}$$

$\vdots$

$\vdots$

$\vdots$

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{5}{2} & \frac{3}{2} \\ 0 & 0 & -\frac{1}{5} \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & \frac{4}{5} & 1 \end{bmatrix}$$

1.3 Ortogonalità

- Due vettori x, y sono ortogonali se $x^T y = \sum_{i=1}^n x_i y_i = 0$;
- Una base $\{v_1, \dots, v_n\}$ è ortogonale se $v_i^T v_j = 0 \quad \forall i \neq j$, se i vettori sono anche unitari la base è detta ortonormale;
- Due sottospazi R e N sono ortogonali se $r^T n = 0 \quad \forall r \in R, n \in N$.

Data una base ortonormale $[q_1, \dots, q_n]$, si può scrivere ogni vettore come:

$$v = c_1 q_1 + \dots + c_n q_n \quad \text{dove } c_i = q_i^T v$$

Definizione Una matrice di proiezione (P) è una matrice quadrata tale che $P = P^2$, se vale $P = P^2 = P^T$ allora è ortogonale.

Definizione Una matrice di rotazione di angolo θ è:

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Definizione Una matrice di riflessione si può ottenere da una matrice di rotazione moltiplicando una delle colonne (o una delle righe) per -1.

Definizione La matrice di Householder è definita come $H = I - 2uu^T$ con u unitario.

Definizione L'ortogonalizzazione di Gram-Schmidt è un procedimento con cui si trasforma un insieme di vettori linearmente indipendenti $\{v_1, v_2, \dots, v_n\}$ in un insieme di vettori ortogonali:

$$u_1 = v_1$$

per $k = 2, \dots, n$:

$$u_k = v_k - \sum_{i=1}^{k-1} \frac{\langle v_k, u_i \rangle}{\langle u_i, u_i \rangle} u_i \quad \langle v_k, u_i \rangle \text{ è il prodotto scalare}$$

1.4 Autovalori e autovettori

1.4.1 Teoria

Definizione Un vettore non nullo x è un autovettore di A se esiste uno scalare λ (detto autovalore) tale che:

$$Ax = \lambda x$$

Si ha:

- $A^k x = \lambda^k x$ per $k = 1, 2, 3, \dots$
- $A^{-1} x = \frac{1}{\lambda} x$ se $\lambda \neq 0$

Data una base di autovettori $[x_1, \dots, x_n]$, si può scrivere ogni vettore come:

$$v = c_1 x_1 + \dots + c_n x_n$$

ed il prodotto Av come:

$$c_1 \lambda_1 x_1 + \dots + c_n \lambda_n x_n.$$

Definizione La traccia di una matrice quadrata ($tr()$) è la somma degli elementi della diagonale principale.

Alcune proprietà:

- Per una matrice quadrata:
 - $\sum_i \lambda_i = \text{tr}(A)$;
 - $\prod_i \lambda_i = \det(A)$.
- Per una matrice simmetrica:
 - tutti gli autovalori sono reali;
 - autovettori associati ad autovalori distinti sono ortogonali.
- $\lambda_1 \neq \lambda_2 \Rightarrow x_1 \cdot x_2 = 0$
- A e BAB^{-1} hanno gli stessi autovalori

1.4.2 Calcolo e diagonalizzazione

Gli autovalori della matrice sono le radici del polinomio caratteristico:

$$\det(A - \lambda I) = 0$$

Una volta trovati ne prendo uno alla volta e risolvo:

$$(A - \lambda_i I)x = 0$$

dove qualunque soluzione non nulla è un autovettore associato.

Definizione La molteplicità algebrica è il numero di volte che l'autovalore compare come radice del polinomio caratteristico.

Definizione La molteplicità geometrica è il numero di autovettori linearmente indipendenti associati all'autovalore.

Vale sempre geometrica $\leq$ algebrica.

Definizione Data una matrice A con n autovettori linearmente indipendenti, costruisco:

$$X = [x_1, \dots, x_n], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$$

con cui posso diagonalizzare A :

$$A = X\Lambda X^{-1}$$

Vale:

$$A^k = X\Lambda^k X^{-1} \text{ con } \Lambda^k = \text{diag}(\lambda_1^k, \dots, \lambda_n^k)$$

Una matrice è diagonalizzabile se possiede n autovettori linearmente indipendenti.

Se esiste un autovalore per cui geometrica $<$ algebrica la matrice non è diagonalizzabile.

Esempio:

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

Il polinomio caratteristico è:

$$\det(A - \lambda I) = \begin{vmatrix} 4 - \lambda & 1 & 0 \\ 0 & 2 - \lambda & 0 \\ 0 & 0 & 3 - \lambda \end{vmatrix}$$

Essendo triangolare:

$$\det(A - \lambda I) = (4 - \lambda)(2 - \lambda)(3 - \lambda) \text{ con } \lambda_1 = 4, \lambda_2 = 2, \lambda_3 = 3$$

Per $\lambda_1 = 4$:

$$(A - 4I)x = 0 \rightarrow \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Da cui ricavo ($y = 0, z = 0$) quindi un autovettore è:

$$v_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Procedendo in modo analogo per gli altri 2:

$$v_2 = \begin{bmatrix} 1 \\ -2 \\ 0 \end{bmatrix}, v_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Costruisco le 2 matrici:

$$P = \begin{bmatrix} 1 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, D = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

Gli autovettori sono linearmente indipendenti quindi posso diagonalizzare:

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & \frac{1}{2} & 0 \\ 0 & -\frac{1}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1.5 Matrici Simmetriche Definite Positive

Definizione Una matrice simmetrica è detta definita positiva se tutti i suoi autovalori sono positivi, se uno o più autovalori sono pari a 0 è detta semidefinita positiva.

Definizione Un minore di una matrice A è il determinante di una sottomatrice quadrata ottenibile da A eliminando alcune righe e/o colonne della stessa, se gli insiemi degli indici delle righe eliminate coincide con quello delle colonne è detto minore principale.

Sono equivalenti le seguenti proprietà:

1. Tutti gli autovalori sono positivi;
2. $x^T S x > 0 \quad \forall x \neq 0$;
3. Tutti i minori principali $D_1, D_2, \dots, D_n$ sono positivi;
4. Tutti i pivot dell'eliminazione di Gauss sono positivi.

Se S è definita positiva $x^T S x = 1$ descrive un'ellisse i cui assi principali, di lunghezza $\frac{1}{\sqrt{\lambda_i}}$, coincidono con gli autovettori della matrice.

Definizione La matrice Hessiana è una matrice quadrata che raccoglie le derivate parziali seconde di una funzione scalare di più variabili:

$$H_f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Definizione Il gradiente di una funzione scalare è il vettore delle derivate parziali prime:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

esso individua la direzione di massima crescita della funzione.

Per una funzione $f(x)$ un punto critico soddisfa $\nabla f(x) = 0$, se la matrice Hessiana in quel punto è definita positiva, allora è un minimo locale.

1.6 Norme

Definizione Una norma ($\|\cdot\|$) è una funzione che associa ad ogni vettore (o matrice) un numero reale non negativo con le seguenti proprietà:

1. **Positività**

$$\|x\| \geq 0, \quad \|x\| = 0 \iff x = 0$$

2. **Omogeneità**

$$\|\alpha x\| = |\alpha| \|x\|, \quad \forall \alpha \in \mathbb{R}$$

3. **Disuguaglianza triangolare**

$$\|x + y\| \leq \|x\| + \|y\|$$

Norma Euclidea (ℓ_2)

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Norma ℓ_1

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

Norma ℓ_∞

$$\|x\|_\infty = \max_i |x_i|$$

Norma ℓ_p

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p} \quad 1 \leq p < \infty$$

Definizione Una norma matriciale si può definire come:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

Da cui seguono $\|Ax\| \leq \|A\| \|x\|$ e $\|AB\| \leq \|A\| \|B\|$.

Norma spettrale²

$$\|A\|_2 = \sigma_1$$

Norma ℓ_1

$$\|A\|_1 = \max_j \sum_i |a_{ij}|$$

Norma ℓ_∞

$$\|A\|_\infty = \max_i \sum_j |a_{ij}|$$

Norma ℓ_N^2

$$\|A\|_N = \sum_{i=1}^n \sigma_i$$

Norma ℓ_F^2

$$\|A\|_F = \sqrt{\sum_{i=1}^n \sigma_i^2}$$

Esempi:

$$x = \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\|x\|_1 = |1| + |-2| + |2| = 5$$

$$\|x\|_2 = \sqrt{1^2 + (-2)^2 + 2^2} = 3$$

$$\|x\|_\infty = 2$$

$$\|x\|_p = (1^3 + 2^3 + 2^3)^{1/3} = 17^{1/3} \quad (\text{con } p = 3)$$

$$\|A\|_1 = |2| + |4| = 6$$

$$\|A\|_\infty = |3| + |4| = 7$$

² σ_i è l' i -esimo valore singolare, vedere la prossima sezione.

1.7 SVD

Data una matrice $A_{m \times n}$ di rango r , esistono:

- n vettori singolari dx $[v_1, v_2, \dots]$
(Autovettori normalizzati³ di $A^T A$)
- m vettori singolari sx $[u_1, u_2, \dots]$
(Autovettori normalizzati³ di AA^T)
- r valori singolari $[\sigma_1, \sigma_2, \dots]$
(Radici quadrate degli autovalori di $A^T A$)

Raggruppandoli in 3 matrici:

$$U = [u_1, \dots, u_m], \quad V = [v_1, \dots, v_n], \quad \Sigma = \begin{bmatrix} \sigma_1 & & 0 \\ & \ddots & \\ 0 & & \sigma_r \end{bmatrix}$$

Si ottiene la Decomposizione ai valori singolari: $A = U\Sigma V^T$.

Questa fattorizzazione permette di scrivere ogni matrice come somma di matrici di rango uno:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T.$$

Dove i contributi sono ordinati per importanza $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$.

Teorema 3 (Eckart-Young) *Troncando la somma ai primi k termini si ottiene A_k che rappresenta la migliore approssimazione di rango k della matrice A rispetto alla norma spettrale.*

Se A è simmetrica definita positiva $U = V$ e i valori singolari coincidono con gli autovalori.

³Rispetto alla norma euclidea.

Esempio:

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \rightarrow A^T = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

$$A A^T = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix}$$

$$A^T A \text{ ha } \lambda_1 = 45, \lambda_2 = 5 \text{ con } v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$A A^T \text{ ha } u_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}, u_2 = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

Quindi:

$$\begin{aligned} A &= \begin{bmatrix} \frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}} \\ \frac{3}{\sqrt{10}} & -\frac{1}{\sqrt{10}} \end{bmatrix} \begin{bmatrix} \sqrt{45} & 0 \\ 0 & \sqrt{5} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \\ &= \sqrt{45} \begin{bmatrix} \frac{1}{\sqrt{10}} \\ \frac{3}{\sqrt{10}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} + \sqrt{5} \begin{bmatrix} \frac{3}{\sqrt{10}} \\ -\frac{1}{\sqrt{10}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \end{aligned}$$

1.8 Analisi dei componenti principali

Definizione Una matrice di dati in cui una riga è una variabile e una colonna un'osservazione è detta centrata se ad ogni riga viene sottratta la media della stessa.

Definizione Per A centrata, la matrice di covarianza è definita come:

$$\frac{1}{n-1}AA^T$$

che è simmetrica e semidefinita positiva.

La varianza totale del dataset è la somma degli autovalori della matrice di covarianza.

Definizione La PCA è una tecnica di riduzione della dimensionalità che sfrutta la SVD per rappresentare un insieme di dati con un numero ridotto di variabili, preservando la maggior parte dell'informazione contenuta nel dataset.

Applicando la SVD ad una matrice centrata si ha che:

- le colonne di U sono le componenti principali;
- gli autovettori della matrice di covarianza coincidono con le componenti principali;
- i valori singolari misurano l'importanza delle rispettive componenti;
- gli autovalori della matrice di covarianza sono $\lambda_i = \frac{\sigma_i^2}{n-1}$;
- le colonne di V descrivono le coordinate delle osservazioni nel nuovo sistema di riferimento.

La prima componente principale individua la direzione lungo la quale i dati presentano la massima variabilità, la seconda è ortogonale alla prima e descrive la massima variabilità residua, ecc. . .

2 Capitolo II

2.1 Minimi quadrati

Definizione Data la SVD di A , definisco $A^+ = V\Sigma^+U^T$ (Σ^+ si ottiene trasponendo Σ ed elevando tutti i suoi elementi non nulli a -1) come la pseudoinversa di Moore-Penrose.

Il sistema $Ax = b$ potrebbe non avere una soluzione esatta poiché b potrebbe non appartenere allo spazio delle colonne di A . In questo caso l'obiettivo è trovare il vettore x che minimizza $\|Ax - b\|_2^2$.

Definizione Il vettore $e = b - Ax$ è detto residuo.

Per trovare x esistono diversi metodi:

- $x = A^+b$
- $x = (A^T A)^{-1} A^T b$ (Se le colonne di A sono linearmente indipendenti)
- $x_\delta = (A^T A + \delta^2 I)^{-1} A^T b$ (Se $A^T A$ non è invertibile), quando $\delta \rightarrow 0$
 $x_\delta \rightarrow A^+b$
- Se $A = QR$ (dove Q ha colonne ortonormali e R è triangolare superiore)
 $x = R^{-1}Q^T b$

Un metodo per trovare QR è quello di Gram-Schmidt, partendo dalle colonne di A :

$$q_1 = \frac{a_1}{\|a_1\|}, \quad u_k = a_k - \sum_{i=1}^{k-1} (a_k^T q_i) q_i, \quad q_k = \frac{u_k}{\|u_k\|}, \quad r_{ij} = q_i^T a_j$$

3 Capitolo III

3.1 $\Delta A \rightarrow \Delta A^{-1}$

Definizione⁴ Data $M = A - UV^T$ con $A \in \mathbb{R}^{n \times n}$ invertibile e $U, V \in \mathbb{R}^{n \times k}$. Se $I - V^T A^{-1} U$ è invertibile si definisce la formula di Sherman-Morrison-Woodbury come:

$$(A - UV^T)^{-1} = A^{-1} + A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1}$$

Se $k = 1$, $U = u, V = v \in \mathbb{R}^n$ e $1 - v^T A^{-1}u \neq 0$, si ricava la formula di Sherman-Morrison:

$$(A - uv^T)^{-1} = A^{-1} + \frac{A^{-1}uv^T A^{-1}}{1 - v^T A^{-1}u}.$$

Se si conosce A^{-1} entrambe permettono di aggiornare direttamente l'inversa senza doverla ricalcolare da zero. Il costo passa da $O(n^3)$ a $O(n^2k + k^3)$ e risulta conveniente quando $k \ll n$.

Esempio Sherman-Morrison:

$$A = A^{-1} = I, \quad u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad v = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$M^{-1} = I + \frac{uv^T}{1 - v^T u} = I - \frac{1}{4} \begin{bmatrix} 3 & 1 \\ 6 & 2 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & -\frac{1}{4} \\ -\frac{3}{2} & \frac{1}{2} \end{bmatrix}$$

Esempio Sherman-Morrison-Woodbury:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}, \quad U = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$(I_2 - V^T A^{-1} U)^{-1} = \begin{bmatrix} 2 & 0 \\ 0 & \frac{3}{2} \end{bmatrix}$$

$$M^{-1} = A^{-1} + A^{-1}U(I_2 - V^T A^{-1}U)^{-1}V^T A^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}$$

Se ho già risolto $A^T A x = A^T b$ ed arriva una nuova osservazione $r^T x = b_{m+1}$ la nuova matrice diventa $A^T A + r^T r$ e dato che $r^T r$ ha rango 1 la sua inversa si può aggiornare tramite Sherman-Morrison.

⁴Per una perturbazione del tipo $A + UV^T$, si ottiene la formula analoga invertendo i segni visibili.

3.2 Splitting $\ell_1 + \ell_2$

Definizione Per trovare una soluzione sparsa di $Ax = b$ basta trovare x per cui la norma ℓ_1 è minima, questo metodo è detto Basis Pursuit.

Definizione Un altro metodo è LASSO, la cui formulazione è:

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1$$

In generale molti problemi si possono scrivere come:

$$\min_u F_1(u) + F_2(u)$$

dove F_1 contiene termini di tipo ℓ_1 e F_2 di tipo ℓ_2 . I 2 però hanno caratteristiche differenti, quindi l'idea è quella di alternare i passi.

Per LASSO la separazione si fa introducendo una variabile ausiliaria z :

$$\min_{x,z} \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|z\|_1 \text{ con il vincolo } x = z$$

Il problema si risolve poi con ADMM⁵ (Alternating Direction Method of Multipliers) che alterna:

1. La parte ℓ_2

$$x^{k+1} = (A^T A + pI)^{-1} (A^T b + p(z^k - u^k))$$

2. La parte ℓ_1

$$z^{k+1} = S_{\lambda/p}(x^{k+1} + u^k) \text{ con } S_\lambda(v) = \text{sign}(v) \max(|v| - \lambda, 0)$$

E combina i risultati:

$$u^{k+1} = u^k + x^{k+1} - z^{k+1}$$

⁵Vedere Capitolo 2.

4 Capitolo VI

4.1 Problemi di minimo e convessità

Definizione Un insieme K è convesso se il segmento che unisce due punti dell'insieme appartiene ancora all'insieme.

Definizione Una funzione F è convessa se:

$$F(\theta x + (1 - \theta)y) \leq \theta F(x) + (1 - \theta)F(y) \quad 0 < \theta < 1$$

Se la disuguaglianza è stretta la funzione è strettamente convessa.

La convessità garantisce che:

- ogni minimo locale è globale;
- l'insieme delle soluzioni è convesso;
- non possono esistere due minimi locali isolati.

Per trovare un possibile punto di minimo si può applicare il metodo di Newton in modo iterativo:

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$

Definizione La matrice Jacobiana J è una matrice che raccoglie le derivate parziali di una funzione vettoriale:

$$J(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Per problemi di minimi quadrati non lineari si usa l'algoritmo di Levenberg-Marquardt:

$$p_{k+1} = p_k - (J^T J + \lambda I)^{-1} J^T (y - \hat{y})$$

Per $\lambda \rightarrow 0$ coincide con il metodo di Gauss-Newton.

4.2 Moltiplicatori di Lagrange

Definizione Per trovare il minimo in presenza di vincoli (espressi come $A^T x = b$), si introducono i moltiplicatori λ e si definisce la lagrangiana come:

$$L(x, \lambda) = F(x) + \lambda^T (A^T x - b)$$

La soluzione si ottiene imponendo:

$$\nabla_x L = 0, \nabla_\lambda L = 0$$

e mettendo tutte le equazioni ottenute a sistema.

Esempio:

$$F(x, y) = x^2 + y^2 \text{ con } x + y = 1$$

$$L(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

$$\frac{\partial L}{\partial x} = 2x + \lambda = 0, \frac{\partial L}{\partial y} = 2y + \lambda = 0, \frac{\partial L}{\partial \lambda} = x + y - 1 = 0$$

$$\begin{cases} 2x + \lambda = 0 \\ 2y + \lambda = 0 \\ x + y - 1 = 0 \end{cases} \rightarrow \begin{cases} 2x = -\lambda \\ 2y = -\lambda \\ x + y = 1 \end{cases} \rightarrow \begin{cases} 2x = 2y \\ x + y = 1 \end{cases} \rightarrow \begin{cases} x = y \\ x + y = 1 \end{cases} \rightarrow \left(\frac{1}{2}, \frac{1}{2}\right)$$

$$F\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

4.3 Discesa del gradiente

Per trovare il minimo di una funzione differenziabile si può sfruttare il gradiente:

$$x_{k+1} = x_k - s_k \nabla f(x_k)$$

Dato che il gradiente punta nella direzione di massima crescita della funzione andando nella direzione opposta si ha la direzione di massima discesa. Non è tuttavia assicurato che si arrivi al punto di minimo globale.

Per scegliere s_k in modo ottimale si usa l'exact line search:

$$s_k = \arg \min_{s \geq 0} f(x_k - s \nabla f(x_k))$$

che risulta tuttavia molto costoso nella pratica, infatti solitamente si usano metodi di scelta approssimati o valori prefissati.

Per accelerare la convergenza si può usare il Momentum, che tiene traccia delle direzioni precedenti:

$$x_{k+1} = x_k - s z_k \text{ con } z_k = \nabla f(x_k) + \beta z_{k-1} \quad (0 \leq \beta < 1)$$

Una variante di quest'ultimo è l'accelerazione di Nesterov:

$$x_{k+1} = x_k + \beta(x_k - x_{k-1}) - s \nabla f(x_k + \gamma(x_k - x_{k-1}))$$

Per ridurre il costo computazionale si può usare una versione stocastica della discesa (SGD) in cui si usa una stima del gradiente.

Un'estensione di SGD è ADAM:

$$x_{k+1} = x_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon} \text{ con } \hat{m}_k = \frac{\beta_1 m_{k-1} + (1 - \beta_1) \nabla f(x_k)}{1 - \beta_1^k}, \hat{v}_k = \frac{\beta_2 v_{k-1} + (1 - \beta_2) \nabla f(x_k)^2}{1 - \beta_2^k}$$

5 Capitolo 2⁶

Data:

$$L(x, y) = f(x) + y^T(Ax - b)$$

dove si vuole minimizzare $f(x)$, la funzione duale è:

$$g(y) = \inf_x L(x, y)$$

e il problema duale è massimizzare $g(y)$.

Un metodo di risoluzione di questo problema è il Dual Ascent:

$$\begin{aligned} x^{k+1} &= \arg \min_x L(x, y^k) \\ y^{k+1} &= y^k + \alpha_k(Ax^{k+1} - b) \quad (\alpha_k > 0) \end{aligned}$$

che converge sotto opportune ipotesi.

Un miglioramento è il Method of Multipliers⁷:

$$\begin{aligned} L_\rho(x, y) &= f(x) + y^T(Ax - b) + \frac{\rho}{2}\|Ax - b\|_2^2 \quad (\rho > 0) \\ x^{k+1} &= \arg \min_x L_\rho(x, y^k) \\ y^{k+1} &= y^k + \rho(Ax^{k+1} - b) \end{aligned}$$

Se la funzione obiettivo è separabile, lo è anche la Lagrangiana:

$$\begin{aligned} f(x) &= \sum_{i=1}^N f_i(x_i) \rightarrow L(x, y) = \sum_{i=1}^N L_i(x_i, y) \\ x_i^{k+1} &= \arg \min_{x_i} L_i(x_i, y^k), \quad y^{k+1} = y^k + \alpha_k(Ax^{k+1} - b) \end{aligned}$$

Questa formulazione è detta Dual Decomposition e permette di risolvere i sottoproblemi in modo indipendente.

⁶Questo capitolo fa riferimento a “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers” di AA.VV.

⁷ADMM è una variante di questo metodo.

6 Regressione

Dato un insieme di dati $D = \{(x_i, y_i)\}_{i=1}^n$ si vuole trovare una funzione che li approssimi.

Assumendo una relazione lineare tra i dati si ha:

$$\hat{y}_i = b_0 + b_1 x_i$$

Dove i parametri si scelgono in modo da minimizzare la differenza tra dati reali e predetti, normalmente tramite MSE:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Nel caso più generale si usa la versione polinomiale:

$$\hat{y} = \sum_{k=0}^d b_k x^k \text{ con } d \text{ grado del polinomio}$$

Un altro tipo di regressione è quella logistica che viene usata quando i valori assumibili da y sono discreti, nel caso binario:

$$\hat{y}_i = \sigma(b_0 + b_1 x_i + \dots) \text{ con } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Il valore ottenuto è una probabilità, scegliendo una soglia si può decidere la classe:

$$\hat{y}_i = \begin{cases} 1 & \text{se } \hat{y}_i \geq c, \\ 0 & \text{altrimenti} \end{cases}$$

La funzione da minimizzare in questo caso è la Cross-Entropy (binaria):

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

7 Deep Learning

7.1 Reti neurali

Definizione Un neurone artificiale è una funzione che dà in output una combinazione lineare dei suoi input, a cui si aggiunge un bias:

$$\sigma(a^T v + b)$$

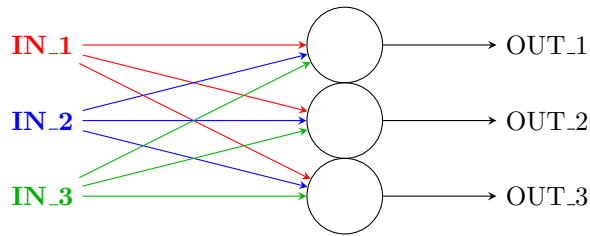
dove

- a è il vettore dei pesi;
- v è il vettore di input;
- b è il vettore dei bias;
- σ è detta funzione di attivazione.

Definizione Un layer è costituito da molti neuroni che elaborano parallelamente lo stesso vettore di ingresso, indicando con $v^{(\ell-1)}$ l'ingresso del layer ℓ :

$$v^{(\ell)} = \sigma(A^{(\ell)} v^{(\ell-1)} + b^{(\ell)})$$

dove A è la matrice dei pesi.



Esempio di layer con 3 neuroni

Mettendo più layer in sequenza si ottiene una rete neurale profonda:

$$v^{(0)} \rightarrow v^{(1)} \rightarrow \dots \rightarrow v^{(L)} = y$$

La rete è quindi una funzione parametrica $y = f(v, X)$ con X insieme di pesi e bias della rete. Il valore y rappresenta una predizione che viene usata in una funzione di costo C per capire quanto si è lontani dal valore reale. L'obiettivo principale è fare in modo che questa distanza sia minima, per fare ciò si usa la discesa del gradiente $X_{k+1} = X_k - \eta \nabla C(X_k)$. Il calcolo del gradiente e l'aggiornamento dei pesi si fa tramite Backpropagation. Nel pratico i dati vengono

divisi in mini-batch⁸ e l'aggiornamento dei parametri avviene dopo l'elaborazione di uno di questi usando la media degli errori ottenuti.

Alcune funzioni comuni:

- Costo

- Quadratico

$$\frac{1}{2} \|y - \hat{y}\|^2$$

- Cross Entropy

$$\sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Attivazione

- Sigmoidale

$$\frac{1}{1 + e^{-x}}$$

- $\tanh$

$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ReLU

$$\max(0, x)$$

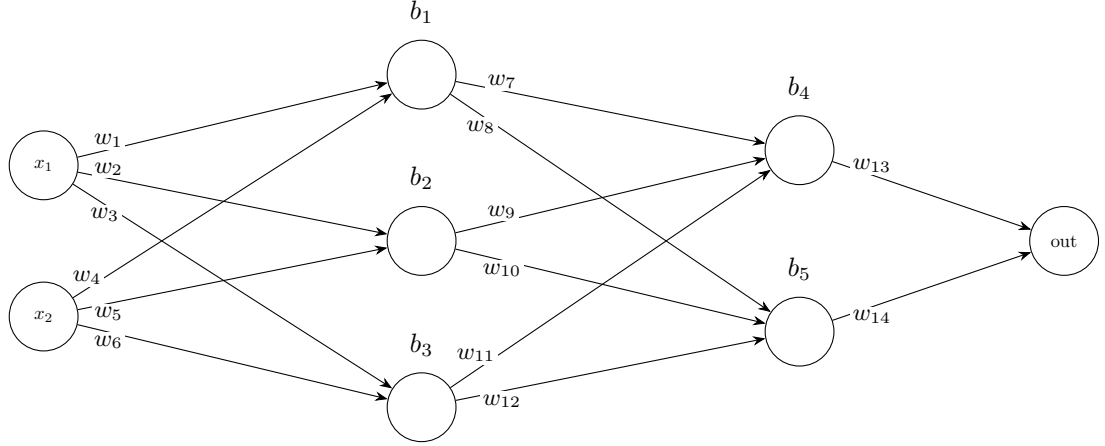
- Softmax

$$\frac{e^{z_i}}{\sum_j e^{z_j}}$$

Quest'ultima si usa nella classificazione multiclasse, dove il target è un vettore di tipo *one-hot*. Viene usata nell'ultimo strato e trasforma il vettore ricevuto in input in un vettore in cui l'*i*-esima riga rappresenta la probabilità di appartenenza all'*i*-esima classe.

⁸L'uso di tutti i mini-batch costituisce un'epoca.

Esempio di aggiornamento (ReLU + Quadratico):



$z_i =$ output neurone i , $a_i = \text{ReLU}(z_i)$

$$a4) \frac{\partial L}{\partial w_6} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_4} \frac{\partial a_4}{\partial z_4} \frac{\partial z_4}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_6}$$

$$a5) \frac{\partial L}{\partial w_6} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_5} \frac{\partial a_5}{\partial z_5} \frac{\partial z_5}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_6}$$

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y, \text{ReLU}'(z) = \begin{cases} 0 & \text{se } z \leq 0 \\ 1 & \text{altrimenti} \end{cases}$$

$$\frac{\partial \hat{y}}{\partial a_4} = w_{13}, \frac{\partial \hat{y}}{\partial a_5} = w_{14}$$

$$\frac{\partial a_4}{\partial z_4} = \text{ReLU}'(z_4), \frac{\partial a_5}{\partial z_5} = \text{ReLU}'(z_5)$$

$$\frac{\partial z_4}{\partial a_3} = w_{11}, \frac{\partial z_5}{\partial a_3} = w_{12}$$

$$\frac{\partial a_3}{\partial z_3} = \text{ReLU}'(z_3)$$

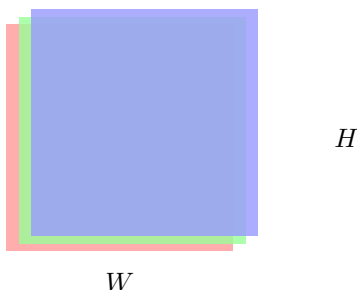
$$\frac{\partial z_3}{\partial w_6} = x_2$$

$$\frac{\partial L}{\partial w_6} = (\hat{y} - y) x_2 \text{ReLU}'(z_3) [w_{13} w_{11} \text{ReLU}'(z_4) + w_{14} w_{12} \text{ReLU}'(z_5)] \rightarrow w_6^{new} = w_6 - \eta \frac{\partial L}{\partial w_6}$$

7.2 CNN

7.2.1 Tensori e operazioni

Un'immagine RGB può essere rappresentata come un tensore $\mathbb{R}^{3 \times H \times W}$:



Prendendo un kernel $\mathbb{R}^{3 \times K \times K}$ si può applicare la generalizzazione della convoluzione di matrici, per ogni posizione:

$$y = \sum_{c=1}^3 \sum_{i=1}^K \sum_{j=1}^K X_c(i, j) K_c(i, j)$$

Utilizzando C' kernel si ottiene un tensore $\mathbb{R}^{C' \times H' \times W'}$ con:

$$H' = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

dove:

- P è il padding usato
- S è lo stride (grandezza del passo)

Esistono alcune varianti:

- **Convoluzione 1×1**

Combina solamente i canali.

- **Grouped Convolution**

I canali si dividono in G gruppi indipendenti, ognuno viene convoluto separatamente e i risultati vengono concatenati.

- **Depthwise Separable Convolution**

Ogni canale ha un suo kernel, i risultati vengono poi concatenati con una 1×1 .

- **Dilated Convolution**

Si sceglie un parametro di dilatazione $r > 1$ che indica la distanza tra un peso ed il suo vicino, per esempio con $r = 2$ c'è una distanza di 1:

$$\begin{bmatrix} a & 0 & b \\ 0 & 0 & 0 \\ c & 0 & d \end{bmatrix}$$

Il pooling è un'operazione che riduce le dimensioni spaziali:

- **Max**

Seleziona l'elemento massimo nella sua finestra.

- **Average**

Calcola la media della sua finestra.

- **Global Max**

Ogni canale viene sostituito dal suo massimo.

- **Global Average**

Ogni canale viene sostituito dalla propria media.

L'operazione inversa è l'upsampling:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{bmatrix} \text{ Nearest Neighbor}$$

$$\rightarrow \begin{bmatrix} 1 & 1.25 & 1.75 & 2 \\ 1.5 & 1.75 & 2.25 & 2.5 \\ 2.5 & 2.75 & 3.25 & 3.5 \\ 3 & 3.25 & 3.75 & 4 \end{bmatrix} \text{ Bilinear interpolation}$$

7.2.2 Funzionamento generale

Una CNN è una rete basata sui layer convoluzionali con un certo numero di filtri (kernel con pesi apprendibili). Un filtro nel pratico è implementato come una matrice di neuroni (con pesi uguali definiti dal filtro) che vede solo una parte di tutti gli input. Per esempio:

$$IN = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, k_1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, k_2 = \begin{bmatrix} 0.5 & 0.6 \\ 0.7 & 0.8 \end{bmatrix}, k_3 = \begin{bmatrix} 1.1 & 1.2 \\ 1.3 & 1.4 \end{bmatrix}, k_4 = \begin{bmatrix} 1.5 & 1.6 \\ 1.7 & 1.8 \end{bmatrix}$$

Con stride 1 e struttura:

$$IN \rightarrow \text{Conv 2 filtri} \rightarrow \text{Conv 2 filtri}$$

Il primo layer ha k_1, k_2 in cui:

- Il primo neurone vede 1,2,4,5
- Il secondo 2,3,5,6
- Il terzo 4,5,7,8
- Il quarto 5,6,8,9

Facendo i calcoli:

$$\begin{aligned} y_{11} &= 1 \cdot 0.1 + 2 \cdot 0.2 + 4 \cdot 0.3 + 5 \cdot 0.4 = 3.7 \\ y_{12} &= 2 \cdot 0.1 + 3 \cdot 0.2 + 5 \cdot 0.3 + 6 \cdot 0.4 = 4.7 \\ y_{13} &= 4 \cdot 0.1 + 5 \cdot 0.2 + 7 \cdot 0.3 + 8 \cdot 0.4 = 6.7 \\ y_{14} &= 5 \cdot 0.1 + 6 \cdot 0.2 + 8 \cdot 0.3 + 9 \cdot 0.4 = 7.7 \end{aligned}$$

$$\begin{aligned} y_{21} &= 1 \cdot 0.5 + 2 \cdot 0.6 + 4 \cdot 0.7 + 5 \cdot 0.8 = 8.5 \\ y_{22} &= 2 \cdot 0.5 + 3 \cdot 0.6 + 5 \cdot 0.7 + 6 \cdot 0.8 = 11.1 \\ y_{23} &= 4 \cdot 0.5 + 5 \cdot 0.6 + 7 \cdot 0.7 + 8 \cdot 0.8 = 16.3 \\ y_{24} &= 5 \cdot 0.5 + 6 \cdot 0.6 + 8 \cdot 0.7 + 9 \cdot 0.8 = 18.9 \end{aligned}$$

Si ottengono le 2 feature map:

$$F_1 = \begin{bmatrix} 3.7 & 4.7 \\ 6.7 & 7.7 \end{bmatrix}, F_2 = \begin{bmatrix} 8.5 & 11.1 \\ 16.3 & 18.9 \end{bmatrix}$$

Il secondo layer ha k_3, k_4 e come input F_1, F_2 . Avendo 2 input i filtri devono avere 2 canali, in questo caso per semplicità hanno gli stessi valori.

$$\begin{aligned} z_1 &= (3.7 \cdot 1.1 + 4.7 \cdot 1.2 + 6.7 \cdot 1.3 + 7.7 \cdot 1.4) \\ &\quad + (8.5 \cdot 1.1 + 11.1 \cdot 1.2 + 16.3 \cdot 1.3 + 18.9 \cdot 1.4) \\ &= 29.58 + 70.62 = 100.20 \end{aligned}$$

$$\begin{aligned} z_2 &= (3.7 \cdot 1.5 + 4.7 \cdot 1.6 + 6.7 \cdot 1.7 + 7.7 \cdot 1.8) \\ &\quad + (8.5 \cdot 1.5 + 11.1 \cdot 1.6 + 16.3 \cdot 1.7 + 18.9 \cdot 1.8) \\ &= 38.06 + 90.78 = 128.84 \end{aligned}$$

Ottenendo:

$$F_3 = [100.20], F_4 = [128.84]$$

7.2.3 Casi d'uso e metriche

- **Classificazione di immagini**

Per creare una CNN di questo tipo vanno convertite tutte le feature map finali in un unico vettore, che viene poi dato come input ad un classificatore “normale”.

Metriche:

- **Matrice di confusione**

Mostra graficamente le decisioni prese⁹:

	Cane	Gatto	Volpe
Cane	47	3	0
Gatto	11	56	0
Volpe	1	0	66

Dove per la classe Cane si hanno 47 scelte giuste (**TP**), 12 falsi positivi (**FP**) e 3 falsi negativi (**FN**).

- **Accuratezza**: $\frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\mathbf{y}_i = \mathbf{y}_i}$ (predizioni corrette)
- **Precisione** (per classe): $Pr = \frac{TP}{TP+FP}$ (affidabilità predizioni)
- **Recall** (per classe): $Re = \frac{TP}{TP+FN}$ (affidabilità riconoscimento)
- **F1**: $2 \frac{PrRe}{Pr+Re}$ (equilibrio Precisione-Recall)

⁹Vedendola una classe alla volta si ha H_0 : immagine non di classe X .

- **Estrazione di caratteristiche**

Se si rimuove la parte che si occupa di classificare da un classificatore si ottiene una CNN in cui l'ultimo layer totalmente connesso genera un vettore (embedding) contenente le caratteristiche dell'immagine. Combinando i risultati si genera uno spazio in cui dato un vettore è possibile cercare i vettori simili che corrispondono a immagini simili, un'applicazione è il riconoscimento facciale.

- **Metric learning**

Si vuole costruire lo spazio di embedding in modo tale che immagini della stessa classe siano vicine e immagini di classi diverse siano lontane. Per farlo si usano delle funzioni di perdita specifiche:

- **Contrastive Loss**

$$\ell(x_i, x_j) = \begin{cases} \|f(x_i) - f(x_j)\|_2^2 & \text{se hanno stessa classe} \\ -\|f(x_i) - f(x_j)\|_2^2 & \text{altrimenti} \end{cases}$$

dove x_i, x_j sono due immagini e $f(\cdot)$ è l'embedding.

- **Triplet Loss**

$$\ell(A, P, N) = \|f(A) - f(P)\|_2^2 < \|f(A) - f(N)\|_2^2$$

dove A è l'immagine da confrontare, P è un'immagine della stessa classe di A e N appartiene ad una classe diversa.

- **Angular Triplet Loss**

$$\ell(A, P, N) = \max\left(0, \tilde{f}(A) \cdot \tilde{f}(N) - \tilde{f}(A) \cdot \tilde{f}(P)\right)$$

dove $\tilde{f}(\cdot)$ è un embedding normalizzato e $\cdot$ è il prodotto scalare.

La rete che impara a costruire questo spazio ha come input (durante l'addestramento) n -uple di immagini, dove n dipende dalla loss.

- **Segmentazione semantica**

Si vuole associare ad ogni pixel dell'immagine una classe, in questo caso si usa una variante delle CNN detta Fully Convolutional Network in cui invece dei layer totalmente connessi si fa upsampling delle feature map in modo da riportarle alle dimensioni dell'immagine di input.

Metriche:

- **IoU** (per classe): $\frac{TP}{TP+FP+FN}$
- **mIoU**: $\frac{1}{K} \sum_{k=1}^K IoU_k$
- **Dice Score** (per classe): $\frac{2TP}{2TP+FP+FN}$
- **Accuratezza pixel**: $\frac{1}{HW} \sum_k TP_k$

7.3 Problemi comuni

- **Sbilanciamento dataset**

- **Inter-classe**

Alcune classi appaiono più volte di altre, risolvibile con la Cross-entropy pesata.

- **Intra-classe**

Ridondanza o presenza di campioni “facili”, risolvibile con la Focal loss.

- **Scala dei dati**

Se il dataset presenta elementi i cui valori sono distribuiti su scale molto diverse possono sorgere problemi (soprattutto durante l'addestramento). Per uniformarli ci sono diversi metodi: normalizzazione¹⁰, mappatura in $[0,1]$, ecc...

- **Overfitting**

Il modello si specializza troppo sui dati di training con conseguente difficoltà a generalizzare, alcune possibili soluzioni sono:

- **Dati di validazione**

Sottoinsieme di dati usati per controllare l'andamento dell'addestramento ma non per l'addestramento in sé, se l'accuratezza di un modello sale ma quella della validazione resta uguale o scende è un segnale di possibile overfitting.

- **Dropout layer**

Il valore di output di ogni nodo del layer precedente viene posto a 0 con una certa probabilità.

- **Batch Normalization layer**

Si calcolano media e varianza¹¹ del batch arrivato come input a questo layer, i valori vengono quindi normalizzati e trasformati:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \rightarrow y_i = \gamma \hat{x}_i + \beta$$

con γ, β apprendibili. Dopo la fine dell'addestramento si usano media e varianza mobili invece di ricalcolare di volta in volta.

¹⁰La normalizzazione non viene fatta considerando il dataset per intero, le 3 parti: training, validazione e test sono normalizzate in modo indipendente tra loro.

¹¹Questo viene fatto in modo indipendente per ogni canale.

7.4 Altri argomenti

- **Transfer learning**

Riutilizzare una rete preaddestrata su un dataset più generale per un nuovo problema più specifico.

- **Linear probing**

Durante il nuovo addestramento gli unici pesi che vengono aggiornati sono quelli del classificatore finale.

- **Fine tuning**

Si aggiornano anche i pesi di alcuni o tutti i layer, generalmente con un learning rate piccolo.

- **Knowledge distillation**

Per addestrare un modello piccolo (*student*) si usa un modello preaddestrato più grande (*teacher*), lo scopo è ottenere un modello che riproduca le probabilità di uscita con costo computazionale inferiore.